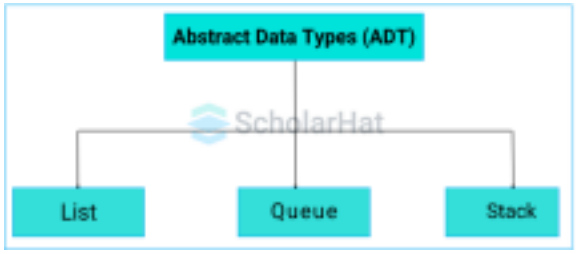
UNIT-3

Abstract Data Types (ADT) and Classes:

An **Abstract Data Type (ADT)** is a high-level concept that defines a data structure by:

- Its **behavior (operations)**
- Its **logical properties**

It does **not specify implementation details**; only *what operations can be performed*.



For example, a **Stack ADT** defines:

- Push (insert element)
- Pop (remove element)
- Peek (view top element)

But it does not specify whether it uses a list, array, or linked list.

Characteristics of ADT

1. **Abstraction** – Only essential features are visible
2. **Encapsulation** – Data and operations are combined
3. **Data Hiding** – Internal implementation is hidden
4. **Modularity** – Easy to manage and modify

Types of ADT

1. Primitive ADT

Basic data types:

- Integer
- Float
- Boolean

2. Non-Primitive ADT

PREPARED BY:PROF.STEFFY MACWAN

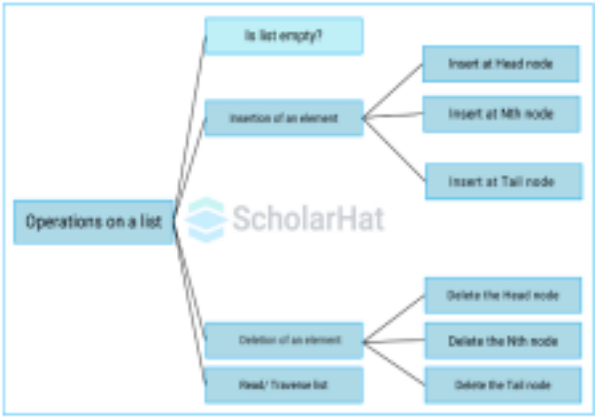
Complex structures:

- List
- Stack
- Queue
- Tree
- Graph

1. List ADT

Lists are linear data structures stored in a non-continuous manner. The list is made up of a series of connected nodes that are randomly stored in the memory. Here, each node consists of two parts, the first part is the data and the second part contains the pointer to the address of the next node.

The first node of a linked list is called the Head, and it acts as an access point. The head pointer points to the first element of the linked list. The last node is called the Tail, and it marks the end of a linked list by pointing to a NULL value.



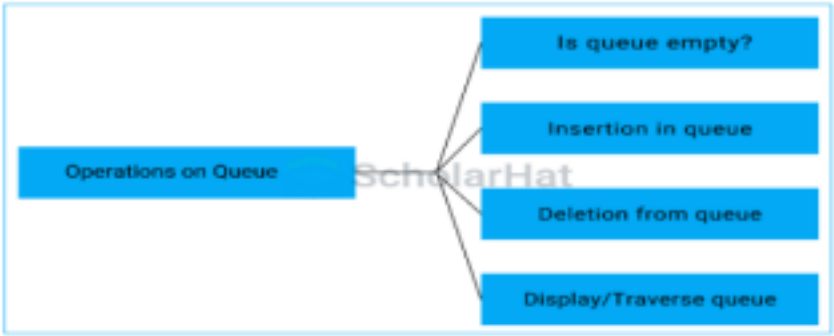
- **front():** returns the value of the node present at the front of the list.
- **back():** returns the value of the node present at the back of the list.
- **push_front(int val):** creates a pointer with value = val and keeps this pointer to the front of the linked list.
- **push_back(int val):** creates a pointer with value = val and keeps this pointer to the back of the linked list.
- **pop_front():** removes the front node from the list.

• **pop_back():** removes the last node from the list.

• **empty():** returns true if the list is empty, otherwise returns false.
• **size():** returns the number of nodes present in the list.

2. Queue ADT

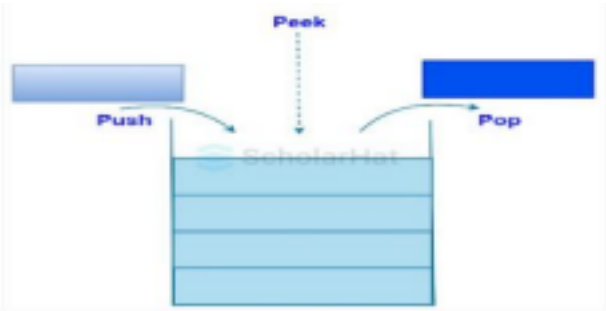
A queue is an ordered list in which insertion is done at one end called REAR and deletion at another end called FRONT. The first inserted element is available first for the operations to be performed and is the first one to be deleted. Hence, it is known as First In First Out, FIFO, or Last In Last Out, LILO.



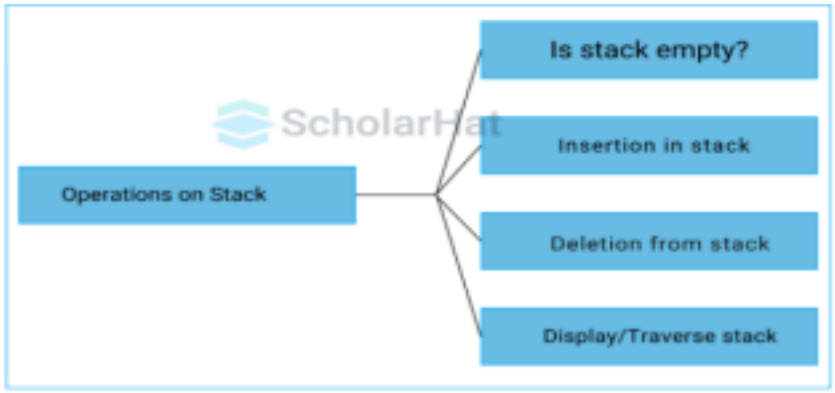
• **enqueue():** Insert an element at the end of the queue.
• **dequeue():** Remove and return the first element of the queue, if the queue isn't empty. • **peek():** Return the element from the queue without removing it.
• **size():** Return the number of elements in the queue.
• **isEmpty():** Return true if the queue is empty, otherwise return false.
• **isFull():** Return true if the queue is full, otherwise return false.

3. Stack ADT:

A stack is an ordered list or we can say a container in which insertion and deletion can be done from the one end known as the top of the stack. The last inserted element is available first and is the first one to be deleted. Hence, it is known as Last In, First Out LIFO, or First In, Last Out FILO.



In a stack, the element at the top is known as the top element. When a new element is added to the stack, it is placed on top of the existing elements. Stacks are dynamic; this means that they do not have a fixed size and their size can be increased or decreased depending upon the number of elements.



• **top():** returns the value of the node present at the top of the stack.
• **push(int val):** creates a node with value = val and puts it at the stack top. •
pop(): removes the node from the top of the stack.
• **empty():** returns true if the stack is empty else false.
• **size():** returns the number of nodes present in the stack

Example Illustrating Stack ADT Operations

• **top():** returns the value of the node present at the top of the stack.
• **push(int val):** creates a node with value = val and puts it at the stack top. •
pop(): removes the node from the top of the stack.
• **empty():** returns true if the stack is empty else false.

- **size():** returns the number of nodes present in the stack

Advantages of ADT

- **Encapsulation:** ADTs help encapsulate data and operations into a single unit, making managing and modifying the data structure easier.
- **Abstraction:** You can work with ADTs without knowing the implementation details, which can simplify programming and reduce errors.

- **Data Structure Independence:** ADTs can be implemented using different data structures, to make it easier to adapt to changing needs and requirements.
- **Information Hiding:** ADTs protect data integrity by controlling access and preventing unauthorized modifications.
- **Modularity:** ADTs can be combined with other ADTs to form more complex data structures increasing flexibility and modularity in programming.

Disadvantages of ADT

- **Overhead:** Implementing ADTs can lead to overhead in terms of memory and processing resulting in reduced performance.
- **Complexity:** ADTs can be complex to implement, especially for large and complex data structures.
- **Learning Curve:** To learn to implement ADTs takes a good amount of time and effort. • **Limited Flexibility:** Some ADTs may be limited in their functionality or may not be suitable for all types of data structures.
- **Cost:** Implementing ADTs may require additional resources and investment increasing the cost of development.

Classes:

A **class** is a blueprint or template used to create objects.
It contains:

- **Variables (data members)**
- **Methods (functions)**

Object

An **object** is an instance of a class.

Example:

obj = ClassName()

Structure of a Class

```
class ClassName:
 def __init__(self, parameters):
 self.variable = parameters

 def method(self):
 # logic
```

Constructor (__init__)

- Special method automatically called when object is created
- Used to initialize object data

Example of Class and Object

```
class Student:
 def __init__(self, name, marks):
 self.name = name
 self.marks = marks

 def display(self):
 print("Name:", self.name)
 print("Marks:", self.marks)
# Creating objects
s1 = Student("Alice", 85)
s2 = Student("Bob", 90)
s1.display()
s2.display()
```

Types of Variables

1. Instance Variables

- Defined inside constructor
- Unique for each object

2. Class Variables

- Shared among all objects

Example:

```
class Student:
 school = "ABC School" # class variable
```

```
def __init__(self, name):
self.name = name # instance variable
```

Types of Methods

1. Instance Method

Uses self

```
def display(self):
 print(self.name)
```


2. Class Method

Uses cls

```
@classmethod
def show_school(cls):
 print(cls.school)
```

3. Static Method

No self or cls

```
@staticmethod
def greet():
 print("Hello")
```

Relationship Between ADT and Class

ADT	Class
Defines what to do	Defines how to do
Logical concept	Actual implementation
	ck operations Python class
	on

ADT = Design
Class = Implementation

Advantages

- Improves code clarity

- Provides data security
- Supports reusability
- Easy debugging and maintenance

Inheritance :

Inheritance is an important concept of **Object-Oriented Programming (OOP)** that allows one class to acquire the properties and methods of another class.

- ➢ The class that inherits is called the **child class (derived class)**
- ➢ The class being inherited from is called the **parent class (base class)**

PREPARED BY:PROF.STEFFY MACWAN

Purpose of Inheritance

- Code reusability
- Reduces redundancy
- Improves readability
- Supports hierarchical classification

Syntax

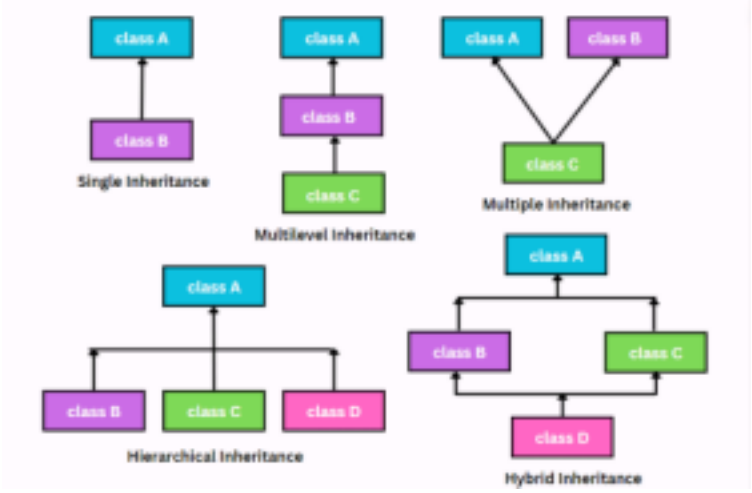
```
class Parent:
 # parent class code

class Child(Parent):
 # child class code
```

Example :

```
class Animal:
 def speak(self):
 print("Animal makes a sound")

class Dog(Animal):
 def bark(self):
 print("Dog barks")

# Create object
d = Dog()

d.speak() # inherited method
```

d.bark() # own method

The Dog class inherits the speak() method from Animal.



1. Single Inheritance

Single inheritance occurs when **one child class inherits from one parent**

class. **Example**

```
class Parent:
 def show(self):
 print("This is Parent class")

class Child(Parent):
 def display(self):
 print("This is Child class")

obj = Child()
obj.show()
obj.display()
```

Output

```
This is Parent class
This is Child class
```

Explanation

The child class can use both its own method and the parent class

method. **2. Multiple Inheritance**

Multiple inheritance occurs when **one child class inherits from more than one parent**

class. **Example**

```
class A:
 def funA(self):
 print("Class A method")

class B:
 def funB(self):
 print("Class B method")

class C(A, B):
 def funC(self):
 print("Class C method")
```



```
obj = C()
obj.funA()
obj.funB()
obj.funC()
```

Output

```
Class A method
Class B method
Class C method
```

Explanation

Class C inherits features from both class A and class B.

3. Multi-Level Inheritance

Multi-level inheritance happens when a class inherits from another derived

class. **Example**

```
class Grandparent:
 def gp(self):
 print("Grandparent class")

class Parent(Grandparent):
 def p(self):
```

```
print("Parent class")

class Child(Parent):
 def c(self):
 print("Child class")

obj = Child()
obj.gp()
obj.p()
obj.c()
```

Output

```
Grandparent class
Parent class
Child class
```

Explanation

Inheritance flows in levels: Grandparent → Parent → Child.

4. Hybrid Inheritance

Hybrid inheritance is a **combination of two or more types of inheritance** (multiple and multilevel). **Example**

```
class A:
 def methodA(self):
 print("Class A")

class B(A):
 def methodB(self):
 print("Class B")

class C(A):
 def methodC(self):
 print("Class C")

class D(B, C):
 def methodD(self):
 print("Class D")
```

```
obj = D()
obj.methodA()
obj.methodB()
obj.methodC()
obj.methodD()
```

Output

```
Class A
Class B
Class C
Class D
```

Explanation

Class D inherits from B and C, which both inherit from A.

Type	Description
Single Inheritance	One parent → one child

Type	Description
Multiple Inheritance M	Multiple parents → one
Inheritance Inheritanc	child
levelsHybrid Inheritan	Combination of inheritance types

ADVANCED CLASS :

1. Method Overloading

Definition

Method overloading means defining multiple methods with the same name but different parameters. Python does not support traditional overloading directly, but it can be achieved using **default arguments or variable arguments**.

Example

```
class Demo:
 def add(self, a=None, b=None, c=None):
 if a and b and c:
 return a + b + c
 elif a and b:
 return a + b
 else:
 return a

obj = Demo()
print(obj.add(5))
print(obj.add(5,10))
print(obj.add(5,10,15))
```

Output

```
5
15
30
```

2. Method Overriding

Definition

- Method overriding occurs when a child class defines a method with the same name as in the parent class.
- The child class method replaces the parent class method.
- It is used in inheritance.

Example

```
class Parent:
 def show(self):
 print("This is Parent class method")
class Child(Parent):
 def show(self): # overriding

        super().show()
 print("This is Child class method")
obj = Child()
obj.show()
```

Output:

This is Child class method

Explanation:
The Child class overrides the show() method of Parent.
So when we call obj.show(), the child's version runs.

Super()

Definition

super() is a built-in function in Python used to call methods of the parent class inside a child

class. It is mainly used in method overriding and constructors.

Syntax

super().method_name(arguments)

- super() refers to the immediate parent class.
- method_name is the method defined in the parent class.

Example of Overriding Without super()

```
class Parent:
 def show(self):
 print("Parent method")
```



```
class Child(Parent):
 def show(self):
 print("Child method")

obj = Child()
obj.show()
```

Output:

Child method

Explanation:
The child class overrides the parent method. The parent method is completely

hidden. **Example of Overriding With super()**

```
class Parent:
 def show(self):
 print("Parent method")

class Child(Parent):
 def show(self):
 super().show() # calling parent method
 print("Child method")

obj = Child()
obj.show()
```

Output:

```
Parent method
Child method
```

Explanation:

1. Object of Child is created.
2. show() method of Child is called.
3. super().show() calls the parent class method.
4. Then the child's additional code runs.

3. Class Method

Definition

A **class method** works with the class itself rather than object instances. It uses @classmethod decorator and cls parameter.

Example

```
class Student:
 school = "ABC School"

 @classmethod
 def display(cls):
 print(cls.school)

Student.display()
```

Output

ABC School

4. Static Method

Definition

A **static method** does not depend on class or object data.
It uses @staticmethod decorator.

Example

```
class Math:
 @staticmethod
 def add(a, b):
 return a + b

print(Math.add(5,3))
```

Output

8

Advantages of Inheritance

- Code reuse
- Saves time and effort
- Easy maintenance
- Extensibility

DIFFERENCE BETWEEN CLASS METHOD AND STATIC METHOD

Basis	Class Method	Static Method
Definition	A method that is bound to the class and operates on class-level data.	A method that is independent of class and instance, used for general utility tasks.
Declarati	Defined using @classmethod decorator.	Defined using @staticmethod
on	Takes cls as the first parameter, which represents the class.	decorator. Does not take self or cls as

Paramet	Can access and modify class variables and class state.	a parameter.
ers	Dependent on the class (works with class data).	Cannot directly access or modify class or instance variables.
Access	Used when we need to work with class level data or factory methods.	Independent of both class and instance.
Dependen cy		Used for helper or utility functions related to the class.
Usage		red variable for all objects. Performing a rinting a message.
Example		

Encapsulation and Information Hiding:

Encapsulation is an Object-Oriented Programming (OOP) concept that means:
Wrapping data (variables) and methods (functions) into a single unit, i.e., a class.

It helps to **control access** to data and ensures that data is used safely.

Features of Encapsulation

- Combines data and methods in one class
- Restricts direct access to variables
- Improves data security
- Provides controlled access through methods

Example :

```
class Student:
 def __init__(self, name, marks):
 self.name = name # public variable
 self.__marks = marks # private variable

 def get_marks(self):
 return self.__marks

 def set_marks(self, marks):
```



```
 if marks >= 0:
```

```
self.__marks = marks
else:
print("Invalid marks")

# Creating object
s = Student("John", 85)

print(s.name) # accessible
print(s.get_marks()) # access via method

s.set_marks(90)
print(s.get_marks())
```

Here, __marks is **encapsulated** and accessed through methods.

Access Specifiers

Python does not have strict access modifiers like other languages, but it uses naming conventions:

Type	Syntax	Access
Public	name	Accessible everywhere
Protected _name		Accessible within class and subclasses
Private __name		Accessible only within

Information Hiding

➢ **Information Hiding** means:
Restricting direct access to internal data and exposing only necessary details.

It is achieved using **private variables and methods**.

Purpose of Information Hiding

- Protect data from unauthorized access
- Prevent accidental changes
- Improve program security
- Maintain integrity of data

Example of Information Hiding

```
class BankAccount:
```

```
def __init__(self, balance):
self.__balance = balance # private variable
```



```
def deposit(self, amount):
self.__balance += amount

def get_balance(self):
return self.__balance

acc = BankAccount(1000)

acc.deposit(500)
print("Balance:", acc.get_balance())

# print(acc.__balance) Error (hidden data)
```

The balance cannot be accessed directly, ensuring **data protection**.

Difference Between Encapsulation and Information Hiding

Basis	**Encapsulation**	**Information Hiding**
Definition	Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit (class).	Information hiding is the concept of restricting access to certain details of an object.
Focus	Focuses on combining data and methods together.	Focuses on restricting access to internal details.
Purpose	To organize code and improve structure.	To protect data from unauthorized access.
Implementation	Achieved using classes and objects.	Achieved using access modifiers like private, protected, and public.
Accessibility	Data can be accessed through methods (getters/setters).	Direct access to sensitive data is restricted.
Security	Provides basic data protection through structure.	Provides stronger data security by hiding implementation details.
Relationship	Encapsulation is a mechanism to achieve information hiding.	Information hiding is a principle achieved using encapsulation.
Example		Making variables private so they cannot be accessed directly.

	A class containing variables and methods together.	

Advantages

- Improves security
- Enhances data integrity
- Makes code modular

- Easy maintenance and debugging

Mortgages and Extended Example: Banking Application (Case Study) A

mortgage is a type of loan used to purchase property (like a house or land), where:

- The borrower pays money over time (EMI – Equated Monthly Installment) • The bank charges **interest**
- The property is used as **security (collateral)**

Components of Mortgage

- **Principal (P)** → Loan amount
- **Interest Rate (R)** → Annual rate (%)
- **Time (T)** → Duration (years/months)
- **EMI** → Monthly payment

where:

- is the principal amount
- is the monthly interest rate (annual rate / 12)
- is the total number of payments (years * 12)

EMI Formula

$$\text{EMI} = \frac{P \times R \times (1+R)^N}{(1+R)^N - 1}.$$

A mortgage is a loan used to purchase property, typically repaid in fixed monthly payments over a specified period. The key components of a mortgage are:

Example:

- Principal: 200,000
- Annual Interest Rate: 4% (or 0.04)
- Loan Term: 30 years

Steps:

1. Convert annual interest rate to monthly:

$r = \frac{0.04}{12} = 0.00333$

2. Calculate the number of payments:

n=30×12=360

3. Apply the formula:

$$M = 200,000 \times \frac{0.00333(1+0.00333)^{360}}{(1+0.00333)^{360}-1}$$

Case Study: Banking Application:

A banking application allows users to manage their accounts, perform transactions, and check balances. The Python program simulates basic banking operations like account creation, deposits, withdrawals, and balance inquiries.

Key Features**:**

1. **Account Creation** – Users can create a bank account with a unique account number.
2. **Deposit Money** – Users can add funds to their account.
3. **Withdraw Money** – Users can withdraw money with sufficient balance.
4. **Balance Inquiry** – Users can check their account balance.
5. **Transaction History** – A log of deposits and withdrawals.

Python Implementation:

```
class BankAccount:
 def __init__(self, name, balance):
 self.name = name
 self.__balance = balance # encapsulation
```

```
def deposit(self, amount):
self.__balance += amount

def withdraw(self, amount):
if amount <= self.__balance:
self.__balance -= amount
else:
print("Insufficient balance")

def get_balance(self):
return self.__balance

# Inheritance
class LoanAccount(BankAccount):
```



```
def __init__(self, name, balance, loan_amount):  super().__init__(name, balance)
self.loan_amount = loan_amount

# Mortgage class
class Mortgage(LoanAccount):
def __init__(self, name, balance, loan_amount, rate, time):  super().__init__(name, balance,
loan_amount)  self.rate = rate
self.time = time

def calculate_emi(self):
r = self.rate / (12 * 100)
n = self.time * 12
P = self.loan_amount

emi = (P * r * (1 + r)**n) / ((1 + r)**n - 1)  return emi

# Usage
m = Mortgage("Alice", 50000, 2000000, 8, 20)

print("Customer:", m.name)
print("Loan Amount:", m.loan_amount)
print("EMI:", round(m.calculate_emi(), 2))
```

Mortgage EMI Problem

Given:

- Principal (P) = 5,00,000
- Annual Interest Rate = 10%
- Time = 2 years

Formula:

$$EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$$

Step-by-Step Solution:

1. Convert annual rate to monthly rate:

$$r = \frac{10}{12 \times 1} = 0.00833$$

2. Number of months:

$$n = 2 \times 12 = 24$$

3. Substitute values:

$$EMI = \frac{500000 \times 0.00833 \times (1+0.00833)^{24}}{(1+0.00833)^{24} - 1}$$

4. Calculate:

$$EMI \approx 23,072$$

Final Answer:

□ Monthly EMI ≈ □23,072